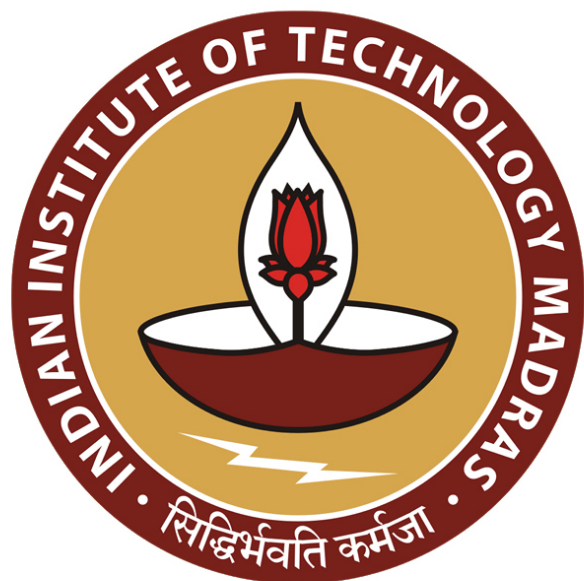
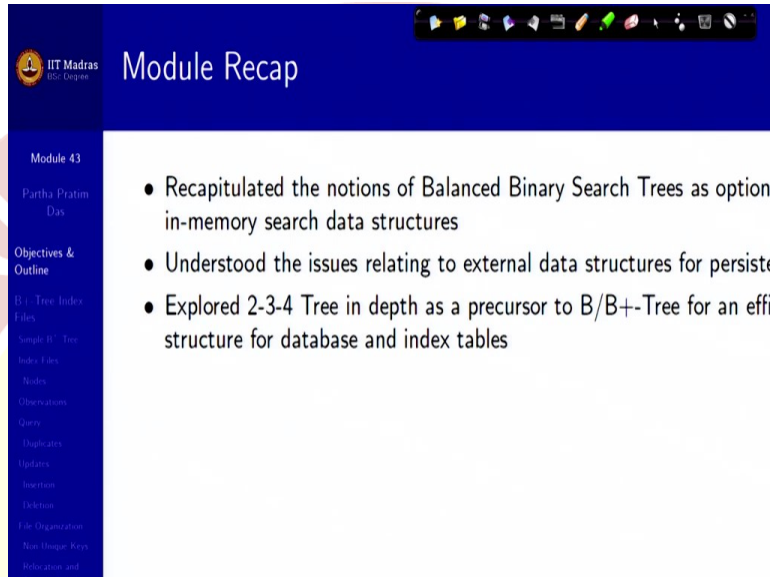




IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Indexing and Hashing/3: Indexing/3

(Refer Slide Time: 00:31)

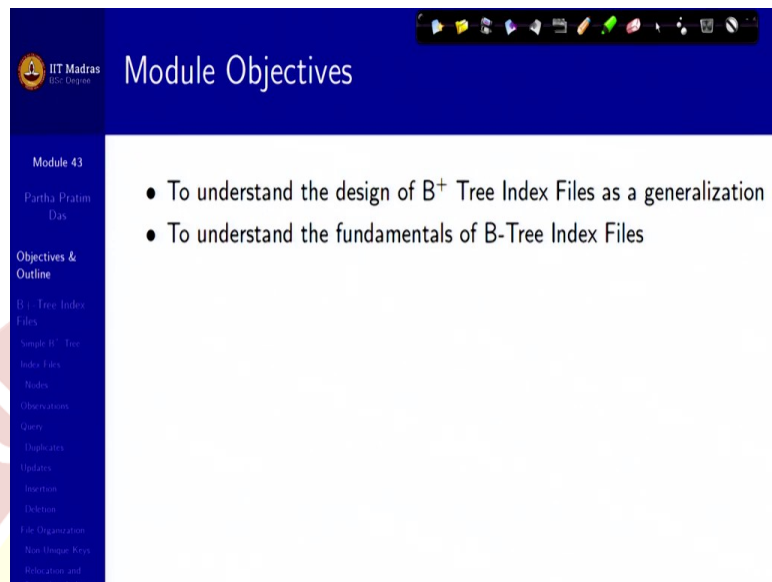


Module Recap

- Recapitulated the notions of Balanced Binary Search Trees as options in-memory search data structures
- Understood the issues relating to external data structures for persistence
- Explored 2-3-4 Tree in depth as a precursor to B/B+-Tree for an efficient structure for database and index tables

Welcome to module 43 of Database Management Systems course in the IIT Madras online B.Sc. program. We have been discussing about indexing and hashing and we have recapitulated binary search trees particularly the balanced ones in the last module, and specifically we took a look at a new interesting data structure called 2-3-4 trees as a precursor to what we are going to discuss today. So, 2-3-4 trees have three different types of nodes and has a unique property that all leaf nodes are at the same level with respect to the root.

(Refer Slide Time: 01:15)



IIT Madras
BSc Degree

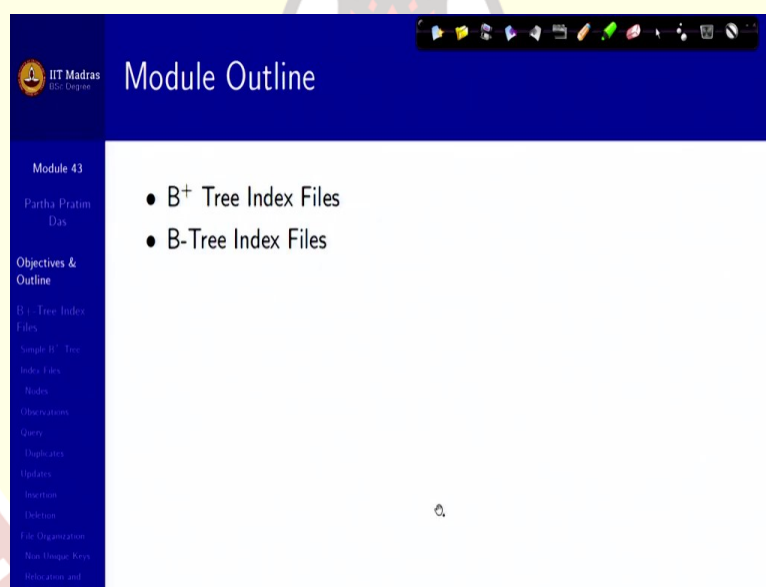
Module Objectives

Module 43
Partha Pratim Das

Objectives & Outline

- B+ Tree Index Files
- Sample B+ Tree
- Index Files
- B-trees
- Observations
- Query
- Duplicates
- Updates
- Insertion
- Deletion
- File Organization
- Non Unique Keys
- Relocation and

- To understand the design of B⁺ Tree Index Files as a generalization of B-Tree Index Files
- To understand the fundamentals of B-Tree Index Files



IIT Madras
BSc Degree

Module Outline

Module 43
Partha Pratim Das

Objectives & Outline

- B+ Tree Index Files
- Sample B+ Tree
- Index Files
- B-trees
- Observations
- Query
- Duplicates
- Updates
- Insertion
- Deletion
- File Organization
- Non Unique Keys
- Relocation and

- B⁺ Tree Index Files
- B-Tree Index Files

So, that is very useful and I, we will extend that idea in terms of B⁺ tree and its use in index file and normal record files and also look at the B tree versions of it.

(Refer Slide Time: 01:32)

B⁺ Tree Index Files

Module 43
Partha Pratim Das
Objectives & Outline
B⁺ Tree Index Files
Simple B⁺ Tree
Index Files
Nodes
Observations
Query
Duplicates
Updates
Insertion
Deletion
File Organization
Non Unique Keys
Indexing and

B⁺ Tree

Module 43
Partha Pratim Das
Objectives & Outline
B⁺ Tree Index Files
Simple B⁺ Tree
Index Files
Nodes
Observations
Query
Duplicates
Updates
Insertion
Deletion
File Organization
Non Unique Keys
Indexing and

The B⁺ Tree

- Is a balanced binary search tree
 - Follows a multi-level index format like 2-3-4 Tree
- Has the leaf nodes denoting actual data pointers
- Ensures that all leaf nodes remain at the same height (like 2-3-4 Tree)
- Has the leaf nodes are linked using a link list
 - Can support random access as well as sequential access
- Example:

Now, coming to B tree B⁺ tree index files first let us, let me quickly introduce through a simple example as to what a B⁺ tree is. So, and we will have more actual data base oriented example later on, but just to define it is a B⁺ tree is a balanced binary search tree, so it will always have a height of some log of the number of nodes the base of that will be decided but it will certainly be more than 2.

So, it is being called a balanced binary search tree because it uses the same concept but it necessarily is not a binary tree, it uses, it follows a multi-level index format like the 2-3-4 tree,

has a leaf nodes denoting the actual data pointers and all leaf nodes are at the same height this is the important thing, all leaf nodes are at the same height.

And the leaf nodes are also linked via a pointer, so that you can, as you can use the indexing structure to access it randomly you can also use it's link structure to for a sequential access, naturally as the construction will ensure that the keys in the leaf node are in sorted order. So, couple of properties but exactly like the 2-3-4 tree only that here we do not have the restriction of having 1 node, 2 node, 3 node, 4 node, all nodes are of the same type, but they may have variable number of data key elements and variable number of pointers.

(Refer Slide Time: 03:43)

IIT Madras
BSc. Computer Science

Module 43
Partha Pratim Das

Objectives & Outline

B+ Tree Index Files

Simple B+ Tree

Index Files

Nodes

Operations

Query

Duplicates

Updates

Insertion

Deletion

File Organization

Heap Image Files

Relocation and

B⁺ Tree (2)

- Internal node contains ✓
 - At least $\frac{n}{2}$ record pointers except the root node ✓
 - At most n pointers
- Leaf node contains
 - At least $\frac{n}{2}$ record pointers and $\frac{n}{2}$ key values
 - At most n record pointer and n key values

IIT Madras
BSc. Computer Science

Module 43
Partha Pratim Das

Objectives & Outline

B+ Tree Index Files

Simple B+ Tree

Index Files

Nodes

Operations

Query

Duplicates

Updates

Insertion

Deletion

File Organization

Heap Image Files

Relocation and

B⁺ Tree (2)

- Internal node contains ✓
 - At least $\frac{n}{2}$ record pointers except the root node ✓
 - At most n pointers
- Leaf node contains
 - At least $\frac{n}{2}$ record pointers and $\frac{n}{2}$ key values
 - At most n record pointer and n key values



IIT Madras
B.Tech. Computer

Module 43

Partha Pratim Das

Objectives & Outline

B+ Tree Index Files

Simple B+ Tree

Index Files

Nodes

Observations

Query

Duplicates

Updates

Insertion

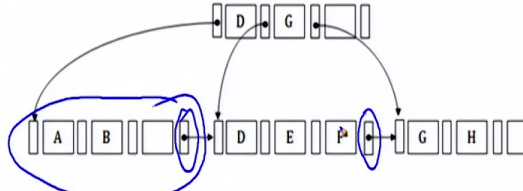
Deletion

File Organization

Non-Linear Files

Index Files and

B⁺ Tree (2)



- Internal node contains
 - At least $\frac{n}{2}$ record pointers except the root node
 - At most n pointers
- Leaf node contains
 - At least $\frac{n}{2}$ record pointers and $\frac{n}{2}$ key values
 - At most n record pointer and n key values

So, this is the basic structure. Now, naturally, we to keep it variable what we do is there is a trade off because the more number of nodes you can keep, I mean more number of keys you can keep in a node will certainly mean that your height gets less obviously, because, so if we but if we have more number of nodes then every node becomes lot more big in size and it may not be possible to bring it to the memory and fit it there, because the computations so it may exceed that block size.

Also, if I have more number of keys in a node then within a node like in the 2-3-4 tree think about a 4 node, there are three keys so multiple comparisons are required sequentially to find out what are the two keys between which your search key is actually to be expected. So, if we increase the number of keys in a node, then that linear such part increases even though the height is reducing.

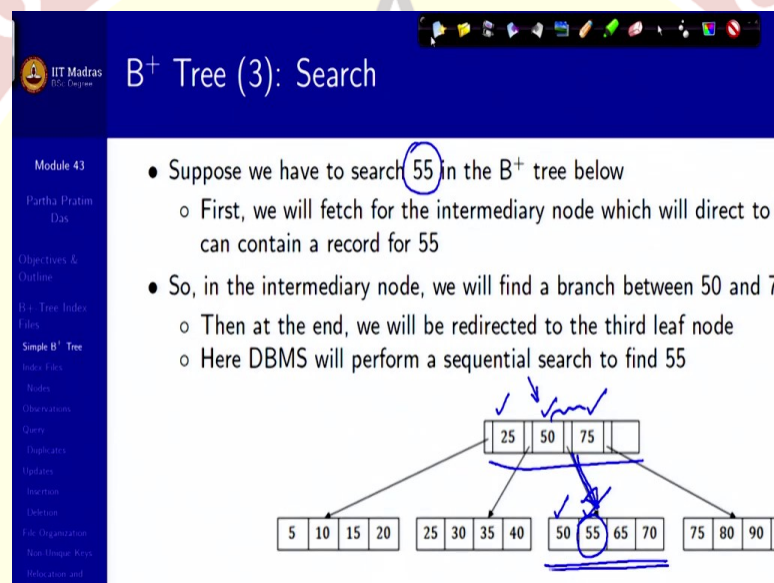
On the other side if we have nodes which have very few keys, then you will need lot more number of nodes and the height will go up. So, to and obviously the storage will also be more. So, to strike a balance we have a very nice rule that in every internal node like this is an internal node, of course this is very special because it is root also, in an internal node at least n by 2 record pointers must be there, where n is a maximum number of keys and pointers that a particular node can fit, so n is that number.

So, of the possible number of record pointers with keys that it can fit it has to be at least n by 2 must be there. And at most naturally, n can be there, so when n are there, then it is totally full,

but it has to be at any point of time at least half filled. Naturally, the exception is the root node, because root node is starting, so at the beginning you may not have enough keys that have gone into the tree, so the root node, it is possible the root node will have very few keys.

In the leaf node also, there has to be n by 2 record pointers and n by 2 key values that finally you have found. Naturally, it will be full, if this value goes up to n , and then it will need say this is a leaf node, so in this you need a additional pointer to the next leaf node for the linear search. So, that is the basic structure of a B+ tree.

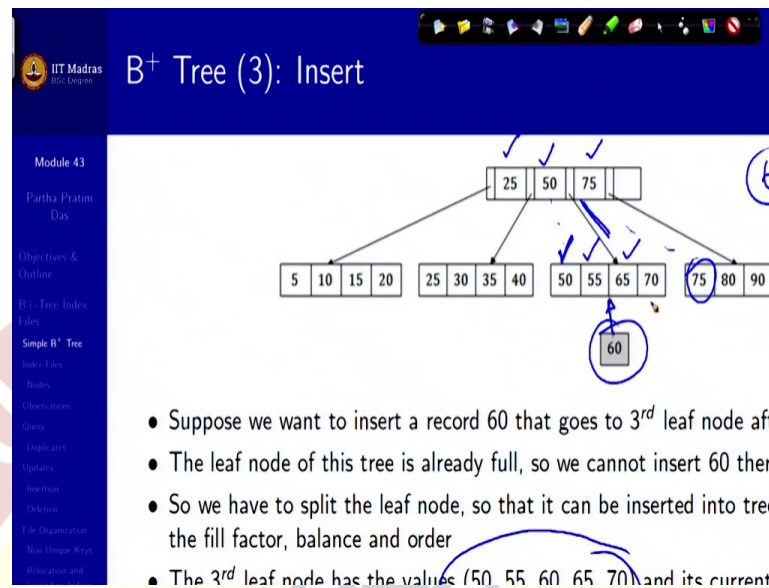
(Refer Slide Time: 06:58)



So, how do you search, very simply as we get in case of 2-3-4 tree, suppose in this given tree you have to search for 55, so I will start with the root, compare with 25 it is more, compare with 50 it is more, compare with 75 it is less, so this is going sequential. So, it is in between these two 55, so I follow the corresponding link.

So, I come to this leaf node and I check with 50 it is more, I check the next one 55 I have found it, so that is the. So, there is one linear here, there is one linear here and then depending on the range you will get to the next level. So, I am just showing one level here. And here, since the key itself is the data, there is no separate pointer in the leaf node to go to the data in general in an index file structure that pointer at the leaf node will also be there.

(Refer Slide Time: 08:00)



So, if I have to insert, what do I have to do? Suppose I have to insert 60, so I will start searching here, this, this, this, and I will find that this is expected here. Then I search here, search here, less, this is more, so I know 60 should go in at this point. And I am deliberately showing a case of a node being full, so this nodes can accommodate, let us say four keys only, and you have a fifth one.

So, it means that this leaf node now have five keys expected and so and its current root is 50 that is from here, like the current root of this is 75, that the least value element in that list value key in that so I will have to split the node as I and split, and do what? Create a new root and push it to the previous level.

(Refer Slide Time: 09:15)

IIT Madras
BSc. Degree

Module 43
Partha Pratim Das

Objectives & Outline

B+ Tree Index Files

Simple B+ Tree

Index Files

Nodes

Operations

Queries

Duplicates

Updates

Insertion

Deletion

File Organization

Non Unique Keys

Reduction and

B⁺ Tree (4): Insert

- So we can group (50, 55) and (60, 65, 70) into 2 leaf nodes
- If these two has to be leaf nodes, the intermediate node cannot branch
- It should have 60 added to it, and then we can have pointers to a new leaf node
- This is how we can insert an entry when there is overflow. In a normal B+ tree, it is very easy to find the node where it fits and then place it in that leaf node

IIT Madras
BSc. Degree

Module 43
Partha Pratim Das

Objectives & Outline

B+ Tree Index Files

Simple B+ Tree

Index Files

Nodes

Operations

Queries

Duplicates

Updates

Insertion

Deletion

File Organization

Non Unique Keys

Reduction and

B⁺ Tree (4): Insert

- So we can group (50, 55) and (60, 65, 70) into 2 leaf nodes
- If these two has to be leaf nodes, the intermediate node cannot branch
- It should have 60 added to it, and then we can have pointers to a new leaf node
- This is how we can insert an entry when there is overflow. In a normal B+ tree, it is very easy to find the node where it fits and then place it in that leaf node

We did all that in the 2-3-4 tree, so we split into two, let us say we are five, so almost half half. And so as we do that 50 is a root here and 60 is the root of the new leaf node that has been created. So, 50 stays at is it were and 60 the new root has to get inserted in the parent which is a root node.

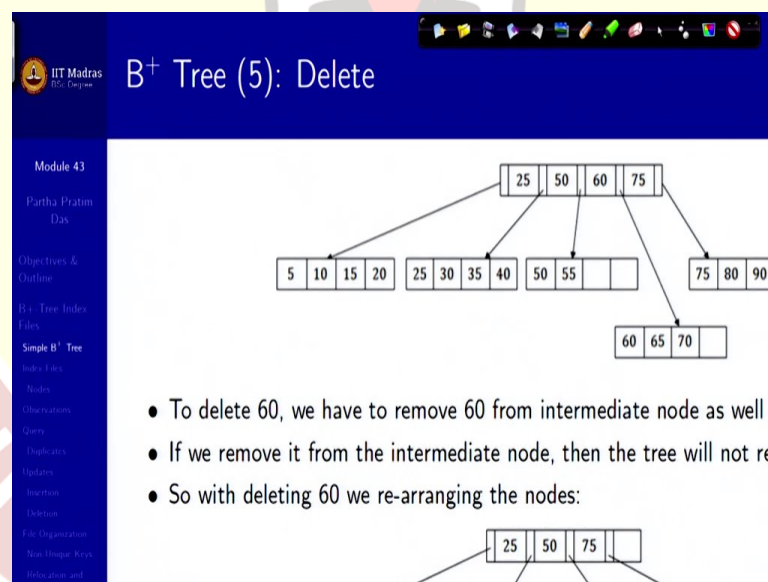
So, earlier we had 25, 50, 75 and null, now you have inserted 60 here which is also a linear insertion, you insert 60 there and make its pointer point to the new leaf node which has so the values have been split in half I said this guarantee, this will guarantee because when are you splitting when the leaf node already has n items.

So, you are trying to insert one more, so you are in n by 2, so what will this necessarily mean that if you split in one you will have n by 2, in the other you will have n by 2 plus 1, so here we have 2, here we have 3 which is and it will always guarantee by going by this root it is always guarantee that a node has at least n by 2 entries.

Similar thing will happen in the internal nodes also, so if I need to insert something which will mean a split for example, if I am trying to insert 85, say I am trying to insert 85, then 85 has to go here between this, so which means this has to now split as 75, 80 and 85, 90, 95 like this, so this 85 has to go into the root node, after 75 but there is no space for that.

So, this has to split again as 25, 50 and as 60, 75, 85. So, I will have a new root node which will have 25 the first key, 60 the second key and we will have null in the rest. And this will point here and this will point here. So, this way it propagates, wherever this happens. So, that is the basic insert process.

(Refer Slide Time: 12:06)

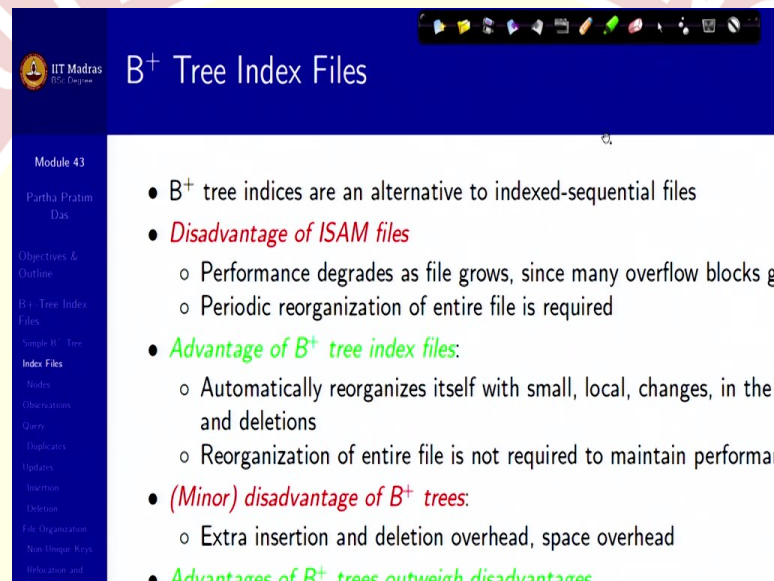


So, as you have the insert you can easily extend this for delete, only thing you will have to do in place of delete is to when you delete you have to first search, delete that and you will have to if it is a root of that leaf node, then it will get deleted up above the whole tree that is one part, and the second is you will have to reorganize because by deletion if a node falls below, your cutoff value you know is n by 2, cutoff value is n by 2, so if it falls below n by 2 you have to check whether it

can be merged with the adjacent leaf node and adjust it, the same thing will happen for the internal nodes also and the height might collapse.

So, you can see that with the height can only increase if I have an insertion and it can only decrease if I have a deletion, in some cases of course not always, oftentimes insertion or deletion will not change the height of the key and the property will be maintained that all leaf nodes are at the same height, that is the basic advantage and that size will be log size.

(Refer Slide Time: 13:23)



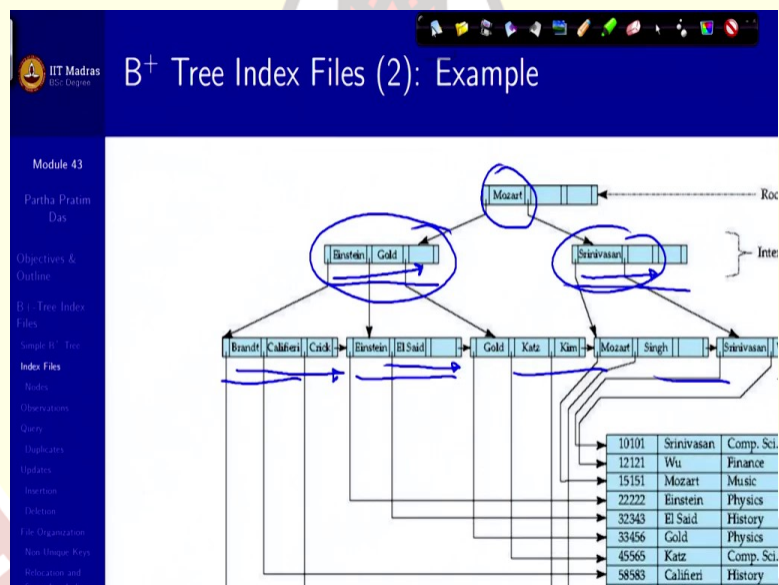
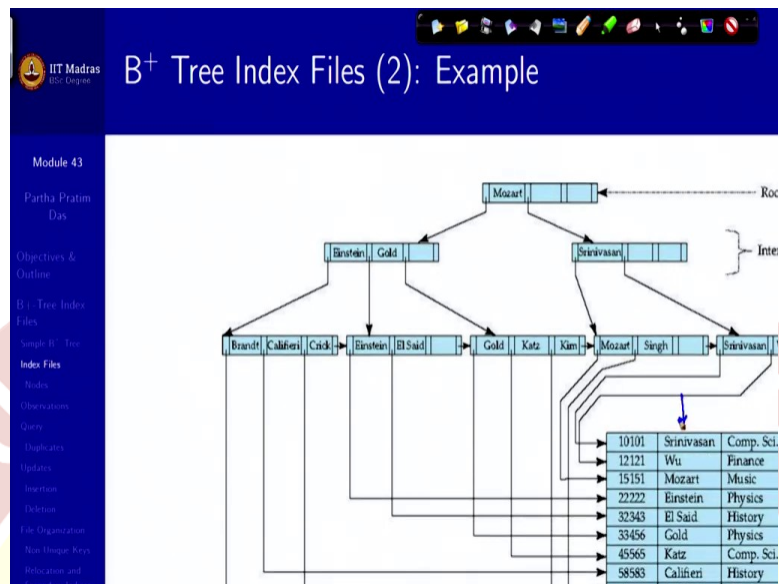
The screenshot shows a presentation slide titled "B⁺ Tree Index Files" from IIT Madras. The slide is part of Module 43, presented by Partha Pratim Das. The left sidebar lists the following topics: Objectives & Outline, B⁺ Tree Index Files, Sample B⁺ Tree Index Files, Index Files, Indexes, Observations, Query, Duplicates, Updates, Insertion, Deletion, File Organization, Non-Unique Keys, and Introduction and Overview. The main content area lists the following points:

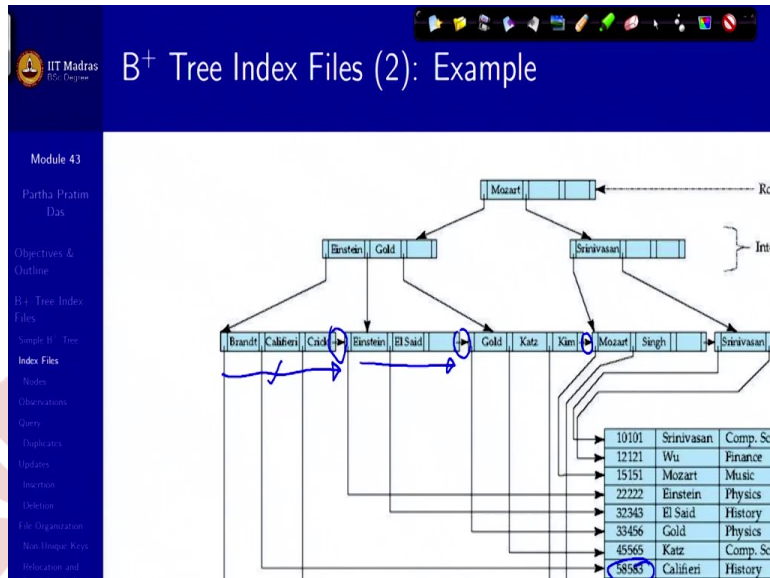
- B⁺ tree indices are an alternative to indexed-sequential files
- **Disadvantage of ISAM files**
 - Performance degrades as file grows, since many overflow blocks get created
 - Periodic reorganization of entire file is required
- **Advantage of B⁺ tree index files:**
 - Automatically reorganizes itself with small, local, changes, in the file and deletions
 - Reorganization of entire file is not required to maintain performance
- **(Minor) disadvantage of B⁺ trees:**
 - Extra insertion and deletion overhead, space overhead
- **Advantages of B⁺ trees outweigh disadvantages**

So, this is what is used for the index files, we will see they are used for the general record files also, so they have the basic advantage is the whatever we saw earlier, the performance of index sequential files we are seeing, they degrade as the file grows, so many overflow blocks are created, reorganization is required which is expensive and so on.

Whereas this one B⁺ tree reorganize itself, it is called a self-organizing data structure, it is not overall organization is never required to maintain the performance. And obviously there are disadvantages of extra insertion, deletion overhead, some space overhead, but it outweighs the, I mean disadvantage is outweighed by the advantages it offers, so it is used very extensively these days.

(Refer Slide Time: 14:24)





So, here is a more, I mean more database kind of example for a B+ tree, you can see this is the data we are trying to keep and where my key being used for this organization is a name. So, we have this organization and the root it is having only Mozart, and these are internal nodes naturally all nodes here will be less than Mozart, all nodes here and subsequently will be more than Mozart these are internal nodes and then finally you have the leaf nodes.

And you can see that in every internal node the keys are sorted, similarly in every leaf node the keys are sorted as well, so if you see in the leaf node, in the internal node it is always going to another internal node, the pointer points to a another internal node or a leaf node, but in case of a leaf node entry, the pointer will point directly to the record data.

So, this is where you will be able to find the data, this the next one points to this data and so on. These are sorted within every leaf node and across the leaf nodes also and you use a extra pointer to link them, so that if you want like select all kind of query where you want to go with the entire file you can, you do not have to search multiple times, you just come to the leftmost one and keep traversing it sequentially. And this is where your actual data is kept. So, this is exactly how and so what I was showing later I have examples with this kind of extensive data also, but I have given you the basic principle of how the B+ tree will work as a indexing file.

(Refer Slide Time: 16:33)

B⁺ Tree Index Files (3): Structure

Module 43
Partha Pratim Das
Objectives & Outline
B⁺ Tree Index Files
Sample B⁺ Tree
Index Files
Nodes
Observations
Query
Duplicates
Updates
Insertion
Deletion
File Organization
Non-Unique Keys
Relocation and

A B⁺ tree is a rooted tree satisfying the following properties:

- All paths from root to leaf are of the same length
- Each node that is not a root or a leaf has between $\lceil \frac{n}{2} \rceil$ and n children
- A leaf node has between an $\lceil \frac{n-1}{2} \rceil$ and $n-1$ values
- Special cases:
 - If the root is not a leaf, it has at least 2 children.
 - If the root is a leaf (that is, there are no other nodes in the tree), between 0 and $(n-1)$ values.

So, this is a formal definition again which I have already stated to you, all paths have same length to the leaf and the leaf has to be at least half, I mean every node has to be at least half filled, except for the naturally for the root node and that keeping that exception aside these conditions will have to satisfy.

(Refer Slide Time: 17:04)

B⁺ Tree Index Files (4): Node Structure

Module 43
Partha Pratim Das
Objectives & Outline
B⁺ Tree Index Files
Sample B⁺ Tree
Index Files
Nodes
Observations
Query
Duplicates
Updates
Insertion
Deletion
File Organization
Non-Unique Keys
Relocation and

- Typical node

P_1	K_1	P_2	...	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	-----	-----------	-----------	-------

- K_i are the search-key values
- P_i are pointers to children (for non-leaf nodes) or pointers to record records (for leaf nodes).
- The search-keys in a node are ordered
 $K_1 < K_2 < K_3 < \dots < K_{n-1}$
(Initially assume no duplicate keys, address duplicates later)

So, at every node what you have? You have pointers and you have keys with them, with keys satisfying this condition for linear search and ordering.

(Refer Slide Time: 17:21)

IIT Madras
B.Tech. Degree

Module 43

Partha Pratim Das

Objectives & Outline

B+ Tree Index Files

Sample B+ Tree

Nodes

Observations

Query

Duplicates

Updates

Insertion

Deletion

File Organization

Non Unique Keys

Reduction and

B⁺ Tree Index Files (5): Leaf Nodes

Properties of a leaf node

- For $i = 1, 2, \dots, n-1$, pointer P_i points to a file record with search-
- If L_i, L_j are leaf nodes and $i < j$, L_i 's search-key values are less than search-key values
- P_n points to next leaf node in search-key order

leaf node

Brand	Califeri	Crick
-------	----------	-------

→ Pointer to next leaf node

10101	Srinivasan	Comp. Sci.	65
12121	Wu	Finance	90
15151	Mozart	Music	40
22222	Einstein	Physics	95
32343	El Said	History	80
33456	Gold	Physics	87
45565	Katz	Comp. Sci.	75
58583	Califeri	History	60

Now, in a leaf node you will have besides you will have other than the key you will have pointers to the data and the pointer to the next leaf node we have already seen this, so these are just more detailed illustrations to show you.

(Refer Slide Time: 17:37)

IIT Madras
B.Tech. Degree

Module 43

Partha Pratim Das

Objectives & Outline

B+ Tree Index Files

Sample B+ Tree

Nodes

Observations

Query

Duplicates

Updates

Insertion

Deletion

File Organization

Non Unique Keys

Reduction and

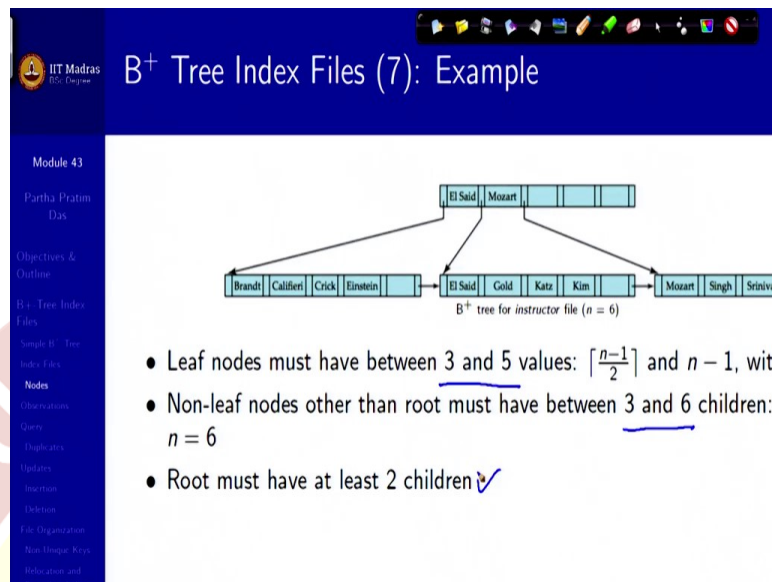
B⁺ Tree Index Files (6): Non-Leaf Nodes

- Non leaf nodes form a multi-level sparse index on the leaf nodes. For with m pointers:
 - All the search-keys in the subtree to which P_1 points are less than
 - For $2 \leq i \leq n-1$, all the search-keys in the subtree to which P_i points are greater than or equal to K_{i-1} and less than K_i
 - All the search-keys in the subtree to which P_n points have values equal to K_{n-1}

P_1	K_1	P_2	...	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	-----	-----------	-----------	-------

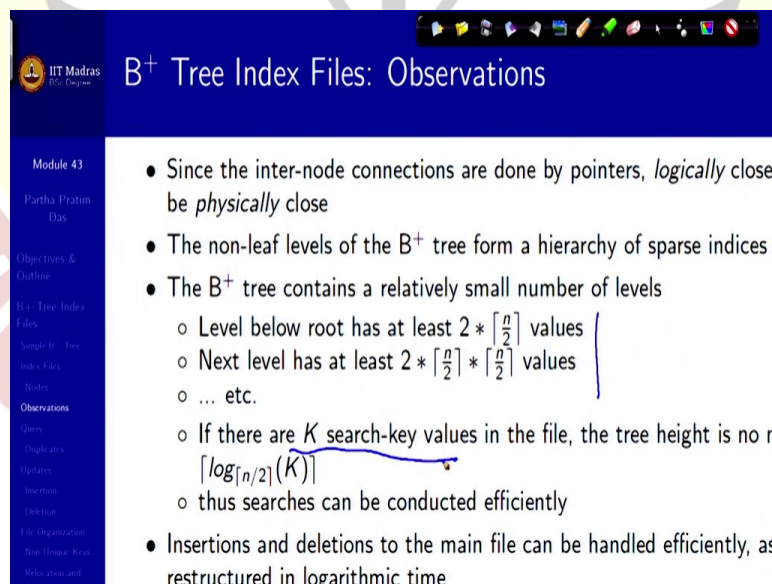
So, in a non-leaf node you will just have the keys and the pointers to other nodes.

(Refer Slide Time: 17:48)



So, here is an example of a B⁺ tree index file, so leaf nodes here are mandated that they must have 3 to 5 values that is we are working with a n value of 6, so internal nodes must have 3 to 6 values, internal node is n by 2 to n , leaf node n by 2 to $n-1$ and the root must have 2 children you can see that all these are satisfied and things are in order.

(Refer Slide Time: 18:26)





B⁺ Tree Index Files: Observations

Module 43
Partha Pratim Das

Objectives & Outline
B⁺ Tree Index Files
Simple B⁺ Tree
Index Files
B⁺ Trees
Observations
Queries
Duplicates
Updates
Insertions
Deletions
File Organization
Non-Unique Keys
Reduction and

- Since the inter-node connections are done by pointers, *logically* close be *physically* close
- The non-leaf levels of the B⁺ tree form a hierarchy of sparse indices
- The B⁺ tree contains a relatively small number of levels
 - Level below root has at least $2 * \lceil \frac{n}{2} \rceil$ values
 - Next level has at least $2 * \lceil \frac{n}{2} \rceil * \lceil \frac{n}{2} \rceil$ values
 - ... etc.
 - If there are K search-key values in the file, the tree height is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$
 - thus searches can be conducted efficiently
- Insertions and deletions to the main file can be handled efficiently, as restructured in logarithmic time



B⁺ Tree Index Files: Observations

Module 43
Partha Pratim Das

Objectives & Outline
B⁺ Tree Index Files
Simple B⁺ Tree
Index Files
B⁺ Trees
Observations
Queries
Duplicates
Updates
Insertions
Deletions
File Organization
Non-Unique Keys
Reduction and

- Since the inter-node connections are done by pointers, *logically* close be *physically* close
- The non-leaf levels of the B⁺ tree form a hierarchy of sparse indices
- The B⁺ tree contains a relatively small number of levels
 - Level below root has at least $2 * \lceil \frac{n}{2} \rceil$ values
 - Next level has at least $2 * \lceil \frac{n}{2} \rceil * \lceil \frac{n}{2} \rceil$ values
 - ... etc.
 - If there are K search-key values in the file, the tree height is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$
 - thus searches can be conducted efficiently
- Insertions and deletions to the main file can be handled efficiently, as restructured in logarithmic time

So, you see that the inter-node connection is being done by the pointer, so whatever is logically close does not need to be physically close, that was becoming that logical closeness is what we need for answering queries, values are close and so on, physical closeness we needed for index sequential file to put the things together here by very efficient use of pointers we are able to get rid of that physical logical contradiction.

So, these are this necessarily if you look at the non-leaf levels, it necessarily conceptually these are basically sparse indices, you can see because you are not keeping everything you are just keeping selective keys and as you go higher and higher you keep more and more sampled key values only as index. So, here some basic arithmetic is given so if there are K -search key values

given that any node must have $n/2$ at least $n/2$ keys, so it will have $n/2$ pointers, that is $n/2$ children at the least it will usually have more.

So, now your base of the logarithm becomes $n/2$, but if it is more, the value actually will be less than K less than this, but this is the highest possible value if all nodes have only $n/2$, then this is for searching K keys where every node can contain n keys and pointers, you will need a height which is $\log_{n/2} K$. So, typically as K will be very large, so will be n , n will not be a very small number like 2, 3, it will be maybe 1000 and so on, so you will get a very efficient height management here, so that is the.

(Refer Slide Time: 20:40)

B⁺ Tree Index Files: Queries

- Find record with search-key value V
 - $C = \text{root}$
 - While C is not a leaf node
 - Let i be least value such that $V \leq K_i$
 - If no such exists, set $C = \text{last non-null pointer in } C$
 - Else { if $(V = K_i)$ Set $C = P_{i+1}$ else set $C = P_i$ }
 - Let i be least value s.t. $K_i = V$
 - If there is such a value i , follow pointer P_i to the desired record
 - Else no record with search-key value k exists

The diagram shows a root node labeled 'Mozart' with three pointers leading to three leaf nodes. The first leaf node contains 'Calderi', 'Einstein', and 'Gold'. The second leaf node contains 'Srinivasan'.

Now, the rest I have given the formal statement of the algorithm in pseudo code of how do you do search which we have already discussed, so I will request you to study the algorithm and convince yourself that it works in a nice way.

(Refer Slide Time: 20:56)

B⁺ Trees Index Files: Queries (2)

- If there are K search-key values in the file, the height of the tree is $n \lceil \log_{\frac{n}{2}}(K) \rceil$
- A node is generally the same size as a disk block, typically 4 kilobytes:
 - and n is typically around 100 (40 bytes per index entry)
- With 1 million search key values and $n = 100$
 - at most $\log_{50}(1,000,000) = 4$ nodes are accessed in a lookup
- Contrast this with a balanced binary tree with 1 million search key values
 - nodes are accessed in a lookup
 - above difference is significant since every node access may need a around 20 milliseconds

So, just to quote some typical values, if we have K as 1 million search keys, I am sorry and n is 100, then you have this logarithm to take 1 million 10 to the power 6 which is taken log of with the base 50, n by 2 and that value comes to 4. So, which means that in a general lookup you will access at most 4 nodes on that tree, quite a manageable ones.

Whereas, if you had a balanced binary tree with 1 million key values, you will require around 20, because 10 to the power 3 is 2 to the power 10, so 10 to the power 6 is 2 to the power 20, so you take a log to the base 2, so you get at least 20, so you have a huge 5 fold improvement here. So, that is a core advantage.

(Refer Slide Time: 22:08)

B⁺ Tree Index Files: Handling Duplicates

- With duplicate search keys
 - In both leaf and internal nodes,
 - ▷ we cannot guarantee that $K_1 < K_2 < K_3 < \dots < K_{n-1}$
 - ▷ but can guarantee $K_1 \leq K_2 \leq K_3 \leq \dots \leq K_{n-1}$
 - Search-keys in the subtree to which P_i points
 - ▷ are $\leq K_i$, but not necessarily $< K_i$,
 - ▷ To see why, suppose same search key value V is present in two L_{i+1} . Then in parent node K_i must be equal to V

Now, naturally there could be duplicates in the search keys, so there could be multiple of having the same value, so we cannot always just guarantee a strict inequality, we will have to go with a relaxed form of inequality and whenever the values are equal, we will have to see how I mean there may be many of them which has to be linearly scanned.

(Refer Slide Time: 22:40)

B⁺ Tree Index Files: Handling Duplicates (2)

- We modify find procedure as follows
 - traverse P_i even if $V = K_i$
 - As soon as we reach a leaf node C check if C has only search key
 - ▷ if so set $C =$ right sibling of C before checking whether C contains V
- Procedure printAll
 - uses modified find procedure to find first occurrence of V
 - Traverse through consecutive leaves to find all occurrences of V

So, we can extend the procedure for search and consequently insert, delete, if you have duplicates in the data.

(Refer Slide Time: 22:51)

 IIT Madras
Dr. Ganga

Updates on B⁺ Trees: Insertion

Module 43

Partha Pratim Das

Objectives & Outline

B⁺ Tree Index Files

Sample B⁺ Tree Index Files

Index

Observations

Query

Duplicates

Updates

Insertion

Deletion

File Organization

Non-Unique Keys

Relocation and

- Find the leaf node in which the search-key value would appear
- If the search-key value is already present in the leaf node
 - Add record to the file
 - If necessary add a pointer to the bucket
- If the search-key value is not present, then
 - Add the record to the main file (and create a bucket if necessary)
 - If there is room in the leaf node, insert (key-value, pointer) pair in
 - Otherwise, split the node (along with the new (key-value, pointer) discussed in the next slide

Now, in terms of updates insertion and deletion, I will not run through the slides, because it is just working out of an insertion example and the deletion example.

(Refer Slide Time: 23:06)

 IIT Madras
Dr. Ganga

Updates on B⁺ Trees: Insertion (2)

Module 43

Partha Pratim Das

Objectives & Outline

B⁺ Tree Index Files

Sample B⁺ Tree Index Files

Index

Observations

Query

Duplicates

Updates

Insertion

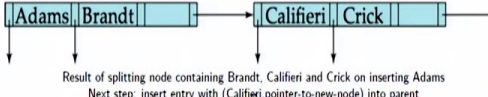
Deletion

File Organization

Non-Unique Keys

Relocation and

- Splitting a leaf node:
 - take the n (search-key value, pointer) pairs (including the one being inserted) in sorted order. Place the first $\lceil \frac{n}{2} \rceil$ in the original node, and the rest in the new node
 - let the new node be p , and let k be the least key value in p . Insert k as a new pointer in the parent of the node being split
 - If the parent is full, split it and **propagate** the split further up
- Splitting of nodes proceeds upwards till a node that is not full is found
- In the worst case the root node may be split increasing the height of the tree



Result of splitting node containing Brandt, Califieri and Crick on inserting Adams
Next step: insert entry with (Califieri, pointer-to-new-node) into parent

Updates on B⁺ Trees: Insertion (3)

Module 43

Partha Pratim Das

Objectives & Outline

B⁺ Tree Index Files

Simple B⁺ Tree

Index Files

Nodes

Operations

Query

Duplicates

Updates

Insertion

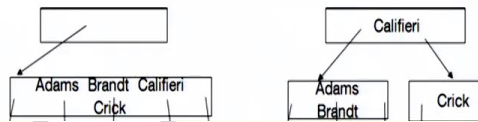
Deletion

File Organization

Non-Unique Keys

Reduction and

- Splitting a non-leaf node: when inserting (k, p) into an already full in
 - Copy N to an in-memory area M with space for $n + 1$ pointers and
 - Insert (k, p) into M
 - Copy $P_1, K_1, \dots, K_{\lceil \frac{n}{2} \rceil - 1}, P_{\lceil \frac{n}{2} \rceil}$ from M back into node N
 - Copy $P_{\lceil \frac{n}{2} \rceil + 1}, K_{\lceil \frac{n}{2} \rceil + 1}, \dots, K_n, P_{n+1}$ from M into newly allocated
 - Insert $(K_{\lceil \frac{n}{2} \rceil}, N')$ into parent N
- Read pseudocode in book!**



Updates on B⁺ Trees: Insertion Example

Module 43

Partha Pratim Das

Objectives & Outline

B⁺ Tree Index Files

Simple B⁺ Tree

Index Files

Nodes

Operations

Query

Duplicates

Updates

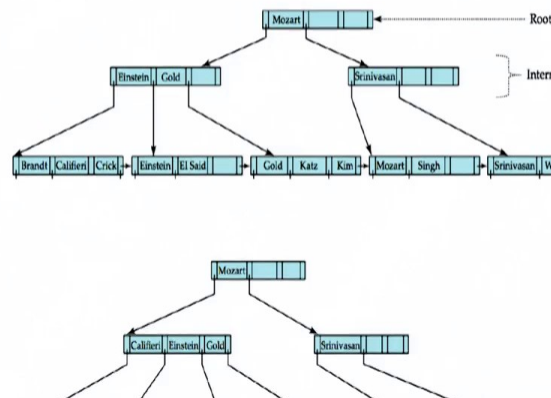
Insertion

Deletion

File Organization

Non-Unique Keys

Reduction and





Updates on B⁺ Trees: Deletion

Module 43

Partha Pratim Das

Objectives & Outline

B⁺ Tree Index Files

Simple B⁺ Tree

Index Files

Nodes

Observations

Query

Duplicates

Updates

Insertion

Deletion

File Organization

New Unique Keys

Redistribution and

- Find the record to be deleted, and remove it from the main file and f present)
- Remove (search-key value, pointer) from the leaf node if there is no l bucket has become empty
- If the node has too few entries due to the removal, and the entries in sibling fit into a single node, then **merge siblings**:
 - Insert all the search-key values in the two nodes into a single node left), and delete the other node.
 - Delete the pair (K_{i-1}, P_i) , where P_i is the pointer to the deleted parent, recursively using the above procedure.



Updates on B⁺ Trees: Deletion (2)

Module 43

Partha Pratim Das

Objectives & Outline

B⁺ Tree Index Files

Simple B⁺ Tree

Index Files

Nodes

Observations

Query

Duplicates

Updates

Insertion

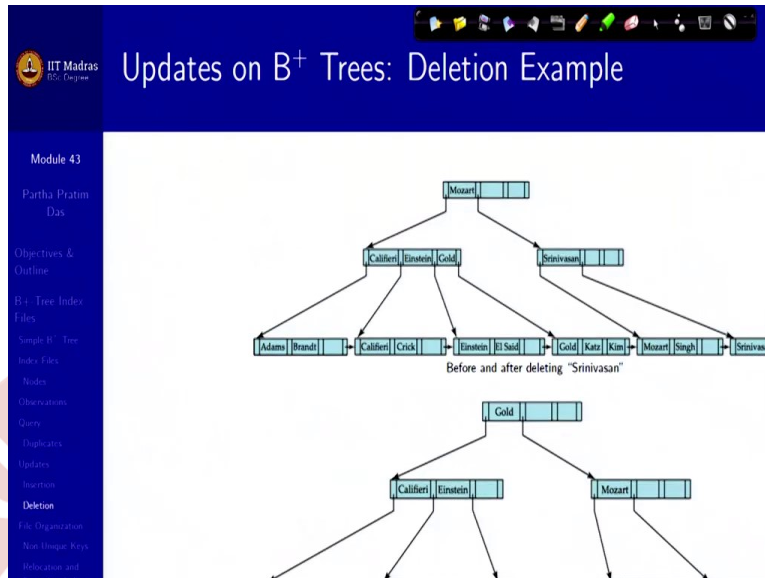
Deletion

File Organization

New Unique Keys

Redistribution and

- Otherwise, if the node has too few entries due to the removal, but th node and a sibling do not fit into a single node, then **redistribute pc**
 - Redistribute the pointers between the node and a sibling such tha than the minimum number of entries
 - Update the corresponding search-key value in the parent of the nc
- The node deletions may cascade upwards till a node which has $\lceil \frac{n}{2} \rceil$ o found
- If the root node has only one pointer after deletion, it is deleted and becomes the root



So, starting from here, we have step by step shown how the different insertion steps will happen, what will, what is being done at every step with these kind of examples, so go through them carefully, and you will understand similar example is also given for deletion. So, you can look at the updates due to deletion through this example. So, to study that and at home and get comfortable with this process.

(Refer Slide Time: 23:42)

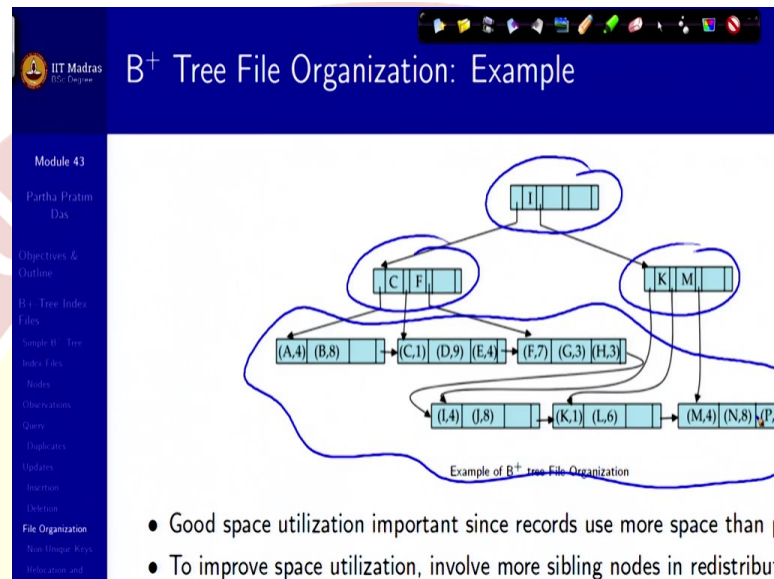
B⁺ Tree File Organization

- Index file degradation problem is solved by using B⁺ Tree indices ✓
- Data file degradation problem is solved by using B⁺ Tree File Organization ✓
- The leaf nodes in a B⁺ tree file organization store records, instead of pointers ✓
- Leaf nodes are still required to be half full ✓
 - Since records are larger than pointers, the maximum number of records stored in a leaf node is less than the number of pointers in a non-leaf node.
- Insertion and deletion are handled in the same way as insertion and deletion in a B⁺ tree index.

But the basic principle as I have explained at the beginning of insert and delete remains the same. So, what it gets us here is the basic index file degradation problem is solved by B tree B⁺ tree indices, data file degradation problem where you actually keep the records can also be handled

by the same, so B+ tree is used it is typically called B+ index tree because it uses a indexing structure, but it can be used for our database index files or it can be used for database record files. And leaf nodes will always need to be half full as we had said.

(Refer Slide Time: 24:31)



So, this is an example of this is the root node, these are internal nodes and this is your leaf where you have the data one after the other.

(Refer Slide Time: 24:48)

Non-Unique Search Keys

- Alternatives to scheme described earlier
 - Buckets on separate block (bad idea)
 - List of tuple pointers with each key
 - ▷ Extra code to handle long lists
 - ▷ Deletion of a tuple can be expensive if there are many duplicates (why?)
 - ▷ Low space overhead, no extra cost for queries
 - Make search key unique by adding a record-identifier
 - ▷ Extra storage overhead for keys
 - ▷ Simpler code for insertion/deletion
 - ▷ Widely used

So, you may have non-unique search keys as I said duplicate, so it will, it might, if you have too many duplicates, then there may be difficulties in handling that, so some people use bucket on

separate block which is I do not think is a great idea, you can have a list of tuple pointers with each key like hanging a kind of chain there, or you can try to forcibly make the search key unique by adding a record pointer, record identifier.

So, that will have overhead of extra storage for keys, but it will make insertion deletion coding simpler and since this search keys are non-unique, it does not really matter which record you pull up or possibly you are trying to pull up all the records, but having that key value. So, if you have this record identifier added to the key then in terms of actual storage, it becomes, it has a overhead of storing that but it makes the things fall into the same structure of insertion deletion and that is why it is quite widely used.

(Refer Slide Time: 26:08)

IIT Madras
Record Relocation and Secondary Indices

Module 43
Partha Pratim Das

- If a record moves, all secondary indices that store record pointers have to be updated
- Node splits in B^+ tree file organizations become very expensive
- **Solution:** Use primary-index search key instead of record pointer in secondary index
 - Extra traversal of primary index to locate record
 - Higher cost for queries, but node splits are cheap
 - Add record-id if primary-index search key is non-unique

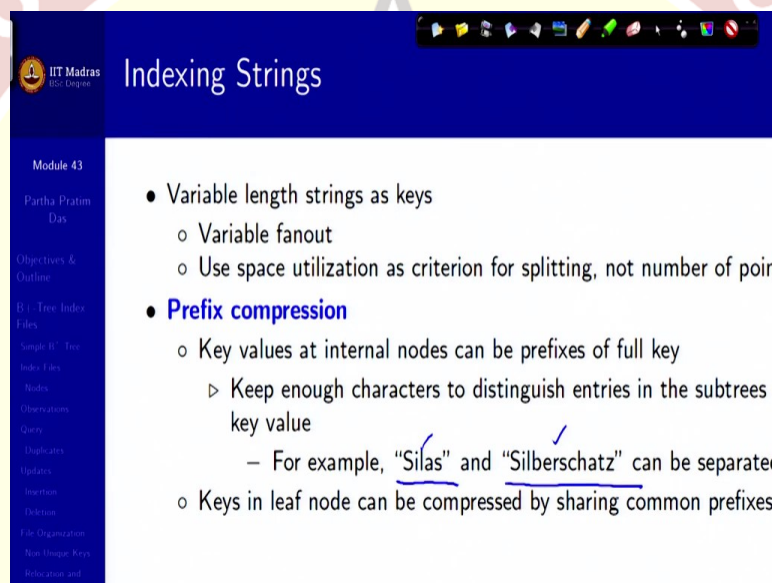
Another issue is if a record relocates, because you are having pointers, finally at the leaf level you are having pointers, so if a record relocates then the stored record pointers have to be updated. And this update is always expensive because it will, it might lead to your node splits which are expensive. So, what is typically done is you use primary index search key instead of when you have a secondary index, I mean this problem primarily happens with the I mean will happen for the secondary index, you have a primary index which actually organizes records.

But you have a secondary index on that. So, if things change in the primary index a record location changes in the primary table, then your secondary index will have to get adjusted because it is using those pointers in turn. So, what you do, you do not use those record pointers

instead you use the primary index search key as if that is the identity, it should be because it is a primary index.

So, that will mean that you have some extra traversal cost, but queries will be a little bit more expensive, but in an overall performance in the amortized performance is often preferred, because node splits are really cheap in this case, they are not as expensive as it would be if records relocate for a in the primary table and you have multiple secondary indices to update. So, these are.

(Refer Slide Time: 28:01)



Indexing Strings

Module 43
Partha Pratim Das

- Variable length strings as keys
 - Variable fanout
 - Use space utilization as criterion for splitting, not number of points
- **Prefix compression**
 - Key values at internal nodes can be prefixes of full key
 - ▷ Keep enough characters to distinguish entries in the subtrees : key value
 - For example, "Silas" and "Silberschatz" can be separated
 - Keys in leaf node can be compressed by sharing common prefixes

Then there is question of how do you index with strings, strings could always be a variable length. So, it has a variable fan out then, you do not know how many will be there. So, one thing which is often done is the prefixing that you use multiple strings and kind of code them in terms of their prefix, the starting value. So, you have Silas you have Silberschatz, you can separate them by Silb and put that as a key. So, this kind of compressions are also possible.

(Refer Slide Time: 28:42)

B-Tree Index Files

Module 43
Partha Pratim Das
Objectives & Outline
B+ Tree Index Files
Sample B+ Tree
Index Files
B-trees
Observations
Query
Duplicates
Updates
Insertion
Deletion
File Organization
Non-Unique Files
Index files and

B-Tree Index Files

- Similar to B^+ tree, but B-tree allows search-key values to appear only once (redundant storage of search keys)
- Search keys in non-leaf nodes appear nowhere else in the B-tree; an a field for each search key in a non-leaf node must be included
- Generalized B-tree leaf node

(a)

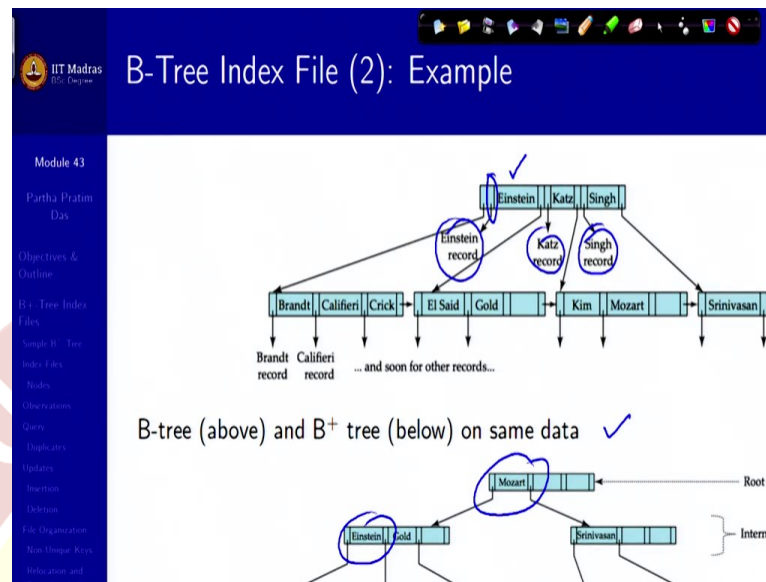
P_1	K_1	P_2	...	P_{n-1}	K_{n-1}	P_n
-------	-------	-------	-----	-----------	-----------	-------

(b)

P_1	B_1	K_1	P_2	B_2	K_2	...	P_{m-1}	B_{m-1}	K_{m-1}	P_m
-------	-------	-------	-------	-------	-------	-----	-----------	-----------	-----------	-------

Now, so that was about B^+ tree there is a compaction on B^+ tree known as B tree. In B tree the basic idea is unlike B^+ tree B tree will allow a search key to appear on the tree only once, once it is in a B^+ tree it will appear multiple times till the root.

(Refer Slide Time: 29:08)



So, you can see the contrast here, this is your B⁺ tree, so you have Mozart happening here then later on happening here, you have Einstein happening here and then happening here and so on. But in a B tree you will have them only once. So, Einstein is here, so Einstein's record in the leaf node will not have another entry.

So, you have a separate pointer right here for the Einstein record, for the Katz record, for the Singh record. So, once a key occurs in a node you keep the record pointer right there, so that it does not occur again, like it is happening here. So, that is a compaction those, only those who have not occurred on any internal node will occur in the leaf node. So, this will naturally take less space to operate.

(Refer Slide Time: 30:17)

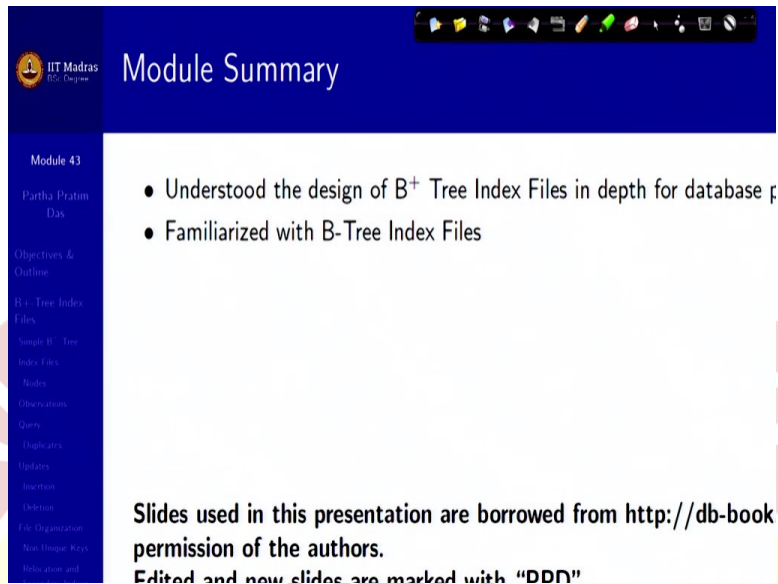
B-Tree Index Files: (3)

- Advantages of B-Tree indices:
 - May use less tree nodes than a corresponding B⁺ Tree
 - Sometimes possible to find search-key value before reaching leaf n
- Disadvantages of B-Tree indices:
 - Only small fraction of all search-key values are found early ✓
 - Non-leaf nodes are larger, so fan-out is reduced. Thus, B-Trees ty greater depth than corresponding B⁺ Tree
 - Insertion and deletion more complicated than in B⁺ Trees
 - Implementation is harder than B⁺ Trees →
- Typically, advantages of B-Trees do not outweigh disadvantages

So, it uses less number of nodes and you are at times be able to find a record even without going to the leaf node, in B⁺ tree you have to go to the leaf node only then you get, but here you might get it earlier. But there are several disadvantages that happened like first of all this advantage of finding a record earlier is not great because only a small fraction of all keys will be on the internal nodes as it is.

And non-leaf nodes are now getting larger, so if the non-leaf node gets larger, the fan out, the number of possible children gets slower, the value of n is decreasing because you have extra information kept in there only, which will mean that your the moment you reduce the fan out here, the height of this B tree will increase. So, also insertion, deletion gets more complicated. So, it is typically that the advantages of B tree do not over outweigh the disadvantages it has. So, the concept has been around it has been used, but it is not a very popular structure.

(Refer Slide Time: 31:34)



Module Summary

- Understood the design of B⁺ Tree Index Files in depth for database p
- Familiarized with B-Tree Index Files

Slides used in this presentation are borrowed from <http://db-book> permission of the authors.
Edited and new slides are marked with "DDDD"

So, this brings us to the end of this module we have discussed about B+ index files for indexing as well as for keeping the primary data and we are familiarized with B tree index files as well. Thank you very much for your attention and we will meet in the next module.